

PROCESSES

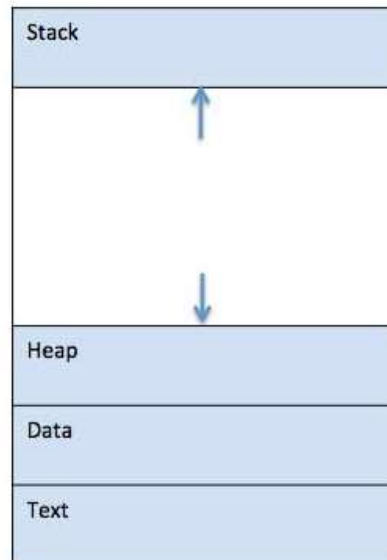
A process is basically a program in execution. The execution of a process must progress in a sequential fashion.

A process is defined as an entity which represents the basic unit of work to be implemented in the system.

To put it in simple terms, we write our computer programs in a text file and when we execute this program, it becomes a process which performs all the tasks mentioned in the program.

- An operating system executes a variety of programs:
 - Batch system – **jobs**
 - Time-shared systems – **user programs** or **tasks**
- Textbook uses the terms **job** and **process** almost interchangeably
- **Process** – a program in execution; process execution must progress in sequential fashion

The program will be stored in the main memory in the following format.



Multiple parts

The program code, also called **text section**

Current activity including **program counter**, processor registers

Stack containing temporary data

Function parameters, return addresses, local variables

Data section containing global variables

Heap containing memory dynamically allocated during run time

Stack

- The process Stack contains the temporary data such as method/function parameters, return address and local variables.

Heap

- This is dynamically allocated memory to a process during its run time.

Text

- This includes the current activity represented by the value of Program Counter and the contents of the processor's registers.

Data

- This section contains the global and static variables.

Relationship between Processes

Concurrent processes running in the operating system may be either “Independent Process” or “Co-operating Process.”

A process is independent if it can not affect or be affected by the other processes executing in the system.

A process that does not share data with any other process is Independent Process.

A process is co-operative, if it can affect or be affected by the other processes executing in the system.

A process that share data with any other process is co-operating process.

Relationship between Processes Contd..

Reasons for process co-operation:

- Information sharing
- Computation speed-up
- Modularity
- Convenience

Information Sharing:

Several users may be interested in same piece of information.

Eg: Shared memory

Must provide environment to allow concurrent access to these types of resources.

Relationship between Processes Contd..

Computation speed-up:

- To run a particular task faster, we must break that it into subtasks.
- Each of these subtasks will execute in parallel with other.
- This is possible only if when computer has multiple processing elements (CPU or I/O channels).

Modularity:

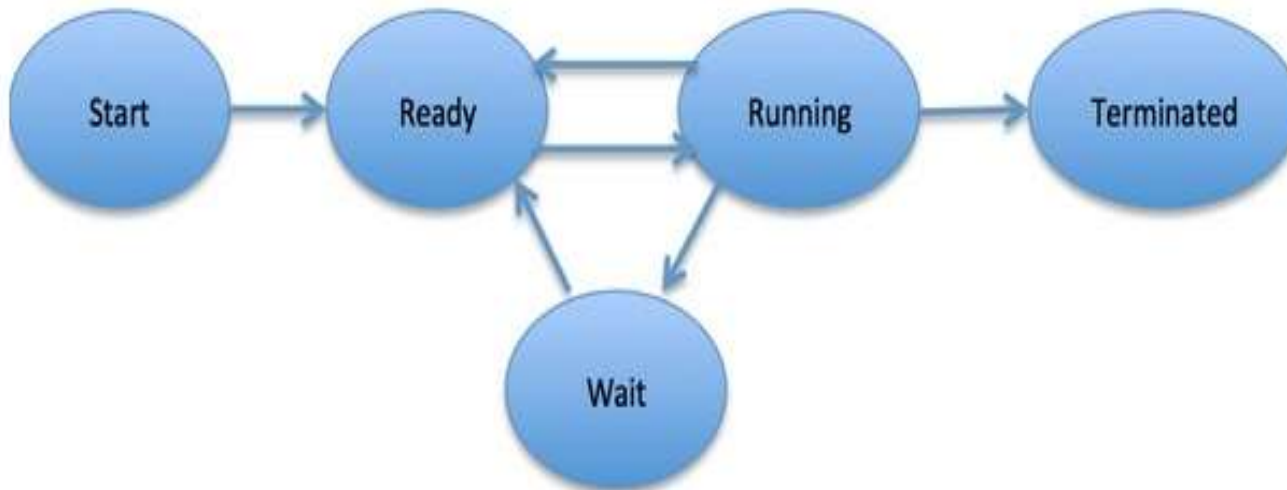
- For co-operating between the systems, we have to construct the system in modular fashion.
- Dividing the system functions into seperate processes or threads.

Convenience:

- Even an individual user may work on many tasks at the same time.
- For instance, a user may be editing, compiling and printing in parallel.

Process Life Cycle

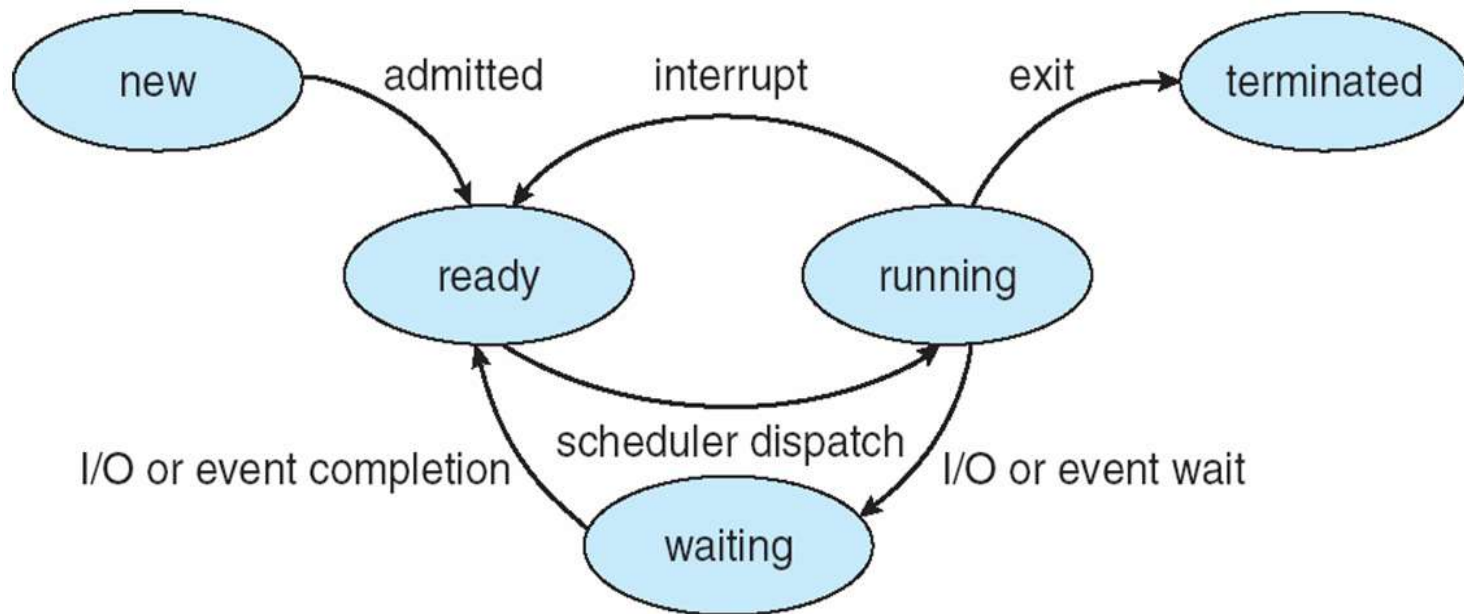
- When a process executes, it passes through different states. These stages may differ in different operating systems, and the names of these states are also not standardized.



Different states of a Process

- As a process executes, it changes **state**
 - **new**: The process is being created
 - **running**: Instructions are being executed
 - **waiting**: The process is waiting for some event to occur
 - **ready**: The process is waiting to be assigned to a processor
 - **terminated**: The process has finished execution

Process State Transitions / Process State Diagram



S.No.	State & Description
1	Start This is the initial state when a process is first started/created.
2	Ready The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after Start state or while running it by but interrupted by the scheduler to assign CPU to some other process.
3	Running Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.
4	Waiting Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.
5	Terminated or Exit Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

PROCESS CONTROL BLOCK / TASK CONTROL BLOCK

A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID).

The architecture of a PCB is completely dependent on Operating System and may contain different information in different operating systems. Here is a simplified diagram of a PCB –



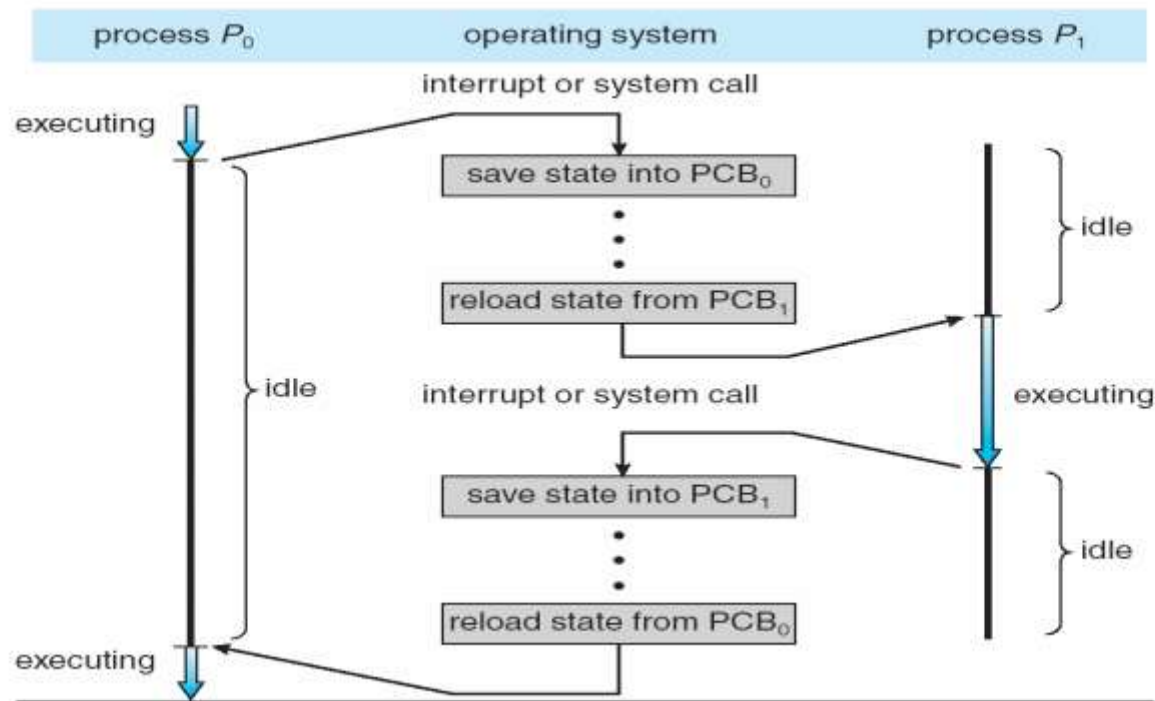
The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminates.

A PCB keeps all the information needed to keep track of a process as listed below in the table –

S.No.	Information & Description
1	Process State The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2	Process privileges This is required to allow/disallow access to system resources.
3	Process ID Unique identification for each of the process in the operating system.
4	Pointer A pointer to parent process.
5	Program Counter Program Counter is a pointer to the address of the next instruction to be executed for this process.
6	CPU registers Various CPU registers where process need to be stored for execution for running state.
7	CPU Scheduling Information Process priority and other scheduling information which is required to schedule the process.
8	Memory management information This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9	Accounting information This includes the amount of CPU used for process execution, time limits, execution ID etc.
10	IO status information This includes a list of I/O devices allocated to the process.

CONTEXT SWITCHING

Context Switching involves storing the context or state of a process so that it can be reloaded when required and execution can be resumed from the same point as earlier. This is a feature of a multitasking operating system and allows a single CPU to be shared by multiple processes.



The above diagram that demonstrates context switching is as follows –

In the above diagram, initially Process 1 is running. Process 1 is switched out and Process 2 is switched in because of an interrupt or a system call. Context switching involves saving the state of Process 1 into PCB1 and loading the state of process 2 from PCB2. After some time again a context switch occurs and Process 2 is switched out and Process 1 is switched in again. This involves saving the state of Process 2 into PCB2 and loading the state of process 1 from PCB1.

Context Switching Triggers: There are three major triggers for context switching. These are given as follows –

- Multitasking:** In a multitasking environment, a process is switched out of the CPU so another process can be run. The state of the old process is saved and the state of the new process is loaded. On a pre-emptive system, processes may be switched out by the scheduler.
- Interrupt Handling:** The hardware switches a part of the context when an interrupt occurs. This happens automatically. Only some of the context is changed to minimize the time required to handle the interrupt.
- User and Kernel Mode Switching:** A context switch may take place when a transition between the user mode and kernel mode is required in the operating system.

The steps involved in context switching are as follows –

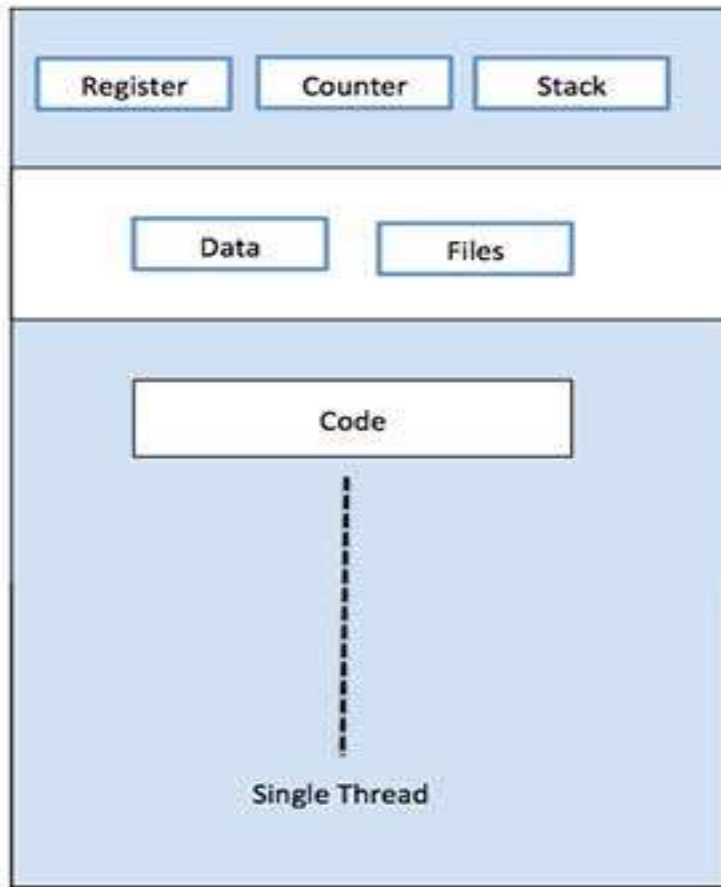
- Save the context of the process that is currently running on the CPU. Update the process control block and other important fields.
- Move the process control block of the above process into the relevant queue such as the ready queue, I/O queue etc.
- Select a new process for execution.
- Update the process control block of the selected process. This includes updating the process state to running.
- Update the memory management data structures as required.
- Restore the context of the process that was previously running when it is loaded again on the processor. This is done by loading the previous values of the process control block and registers.

What is Thread?

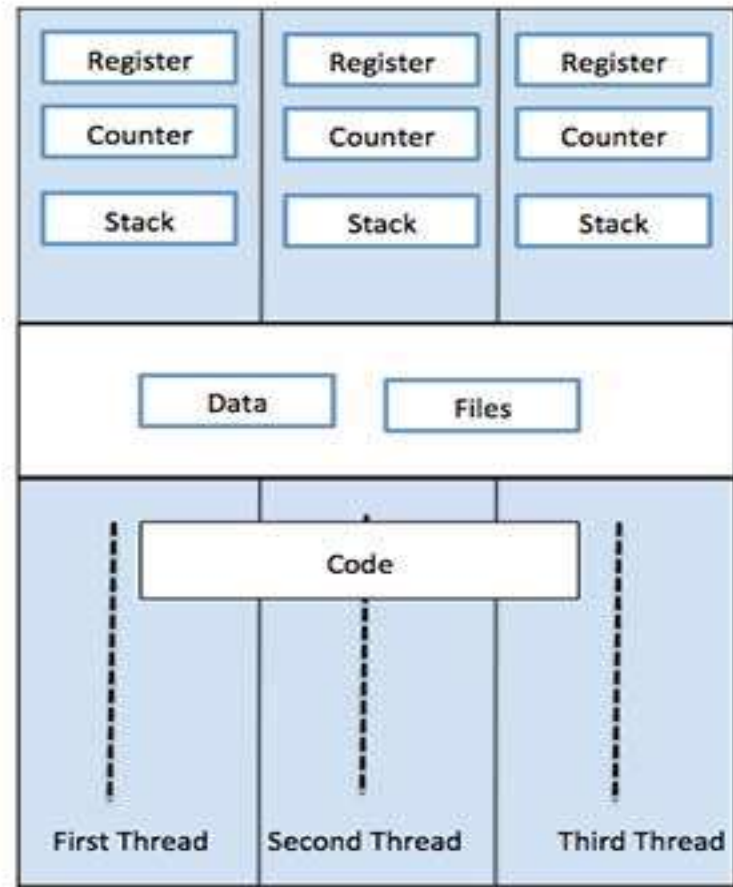
A thread is a flow of execution through the process code, with its own program counter that keeps track of which instruction to execute next, system registers which hold its current working variables, and a stack which contains the execution history.

A thread shares with its peer threads few information like code segment, data segment and open files. When one thread alters a code segment memory item, all other threads see that. A thread is also called a **lightweight process**. Threads provide a way to improve application performance through parallelism. Threads represent a software approach to improving performance of operating system by reducing the overhead thread is equivalent to a classical process.

Each thread belongs to exactly one process and no thread can exist outside a process. Each thread represents a separate flow of control. Threads have been successfully used in implementing network servers and web server. They also provide a suitable foundation for parallel execution of applications on shared memory multiprocessors. The following figure shows the working of a single-threaded and a multithreaded process.



Single Process P with single thread



Single Process P with three threads

Differences between Process and Thread

S.No.	Process	Thread
1	Process is heavy weight or resource intensive.	Thread is light weight, taking lesser resources than a process.
2	Process switching needs interaction with operating system.	Thread switching does not need to interact with operating system.
3	In multiple processing environments, each process executes the same code but has its own memory and file resources.	All threads can share same set of open files, child processes.
4	If one process is blocked, then no other process can execute until the first process is unblocked.	While one thread is blocked and waiting, a second thread in the same task can run.
5	Multiple processes without using threads use more resources.	Multiple threaded processes use fewer resources.
6	In multiple processes each process operates independently of the others.	One thread can read, write or change another thread's data.

Advantages / Benefits of Threads:

- Threads minimize the context switching time.
- Use of threads provides concurrency within a process.
- Efficient communication.
- It is more economical to create and context switch threads.
- Threads allow utilization of multiprocessor architectures to a greater scale and efficiency.

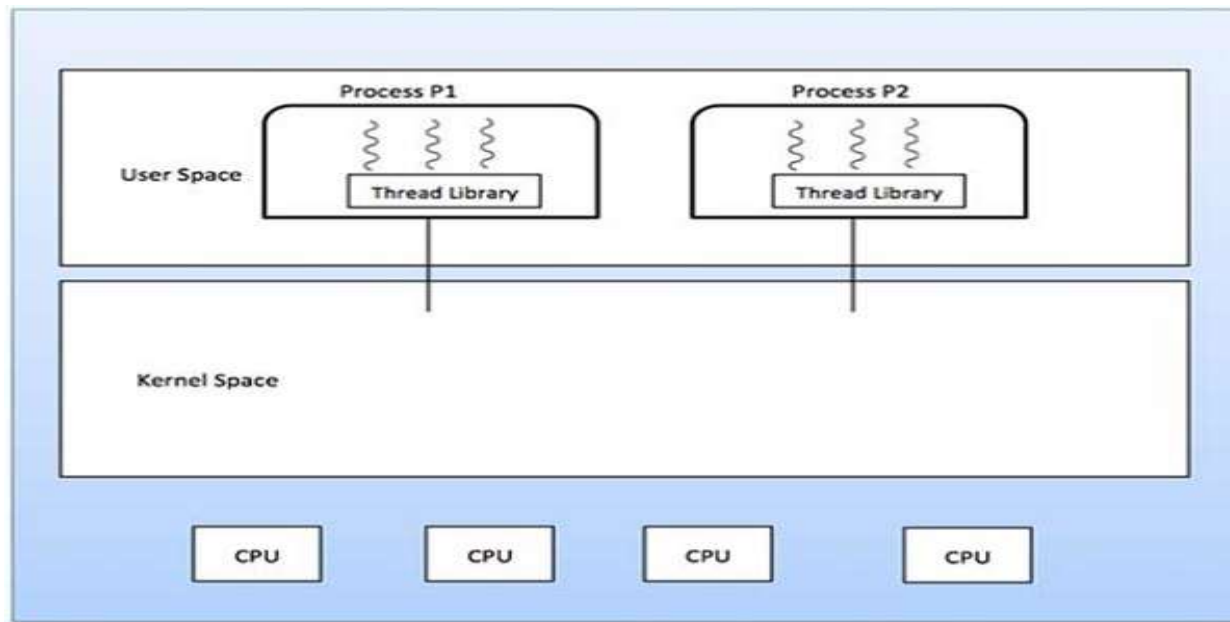
Types of Threads:

Threads are implemented in following two ways –

- **User Level Threads** – User managed threads.
- **Kernel Level Threads** – Operating System managed threads acting on kernel, an operating system core.

User Level Threads:

In this case, the thread management kernel is not aware of the existence of threads. The thread library contains code for creating and destroying threads, for passing message and data between threads, for scheduling thread execution and for saving and restoring thread contexts. The application starts with a single thread.



Advantages

- Thread switching does not require Kernel mode privileges.
- User level thread can run on any operating system.
- Scheduling can be application specific in the user level thread.
- User level threads are fast to create and manage.

Disadvantages

- In a typical operating system, most system calls are blocking.
- Multithreaded application cannot take advantage of multiprocessing.

Kernel Level Threads

In this case, thread management is done by the Kernel. There is no thread management code in the application area. Kernel threads are supported directly by the operating system. Any application can be programmed to be multithreaded. All of the threads within an application are supported within a single process.

The Kernel maintains context information for the process as a whole and for individual threads within the process. Scheduling by the Kernel is done on a thread basis. The Kernel performs thread creation, scheduling and management in Kernel space. Kernel threads are generally slower to create and manage than the user threads.

Advantages

- Kernel can simultaneously schedule multiple threads from the same process on multiple processes.
- If one thread in a process is blocked, the Kernel can schedule another thread of the same process.
- Kernel routines themselves can be multithreaded.

Disadvantages

- Kernel threads are generally slower to create and manage than the user threads.
- Transfer of control from one thread to another within the same process requires a mode switch to the Kernel.

Difference between User-Level & Kernel-Level Thread

S.No.	User-Level Threads	Kernel-Level Thread
1	User-level threads are faster to create and manage.	Kernel-level threads are slower to create and manage.
2	Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
3	User-level thread is generic and can run on any operating system.	Kernel-level thread is specific to the operating system.
4	Multi-threaded applications cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.

Multithreading:

Multithreading allows the execution of multiple parts of a program at the same time. These parts are known as threads and are lightweight processes available within the process. Therefore, multithreading leads to maximum utilization of the CPU by multitasking.

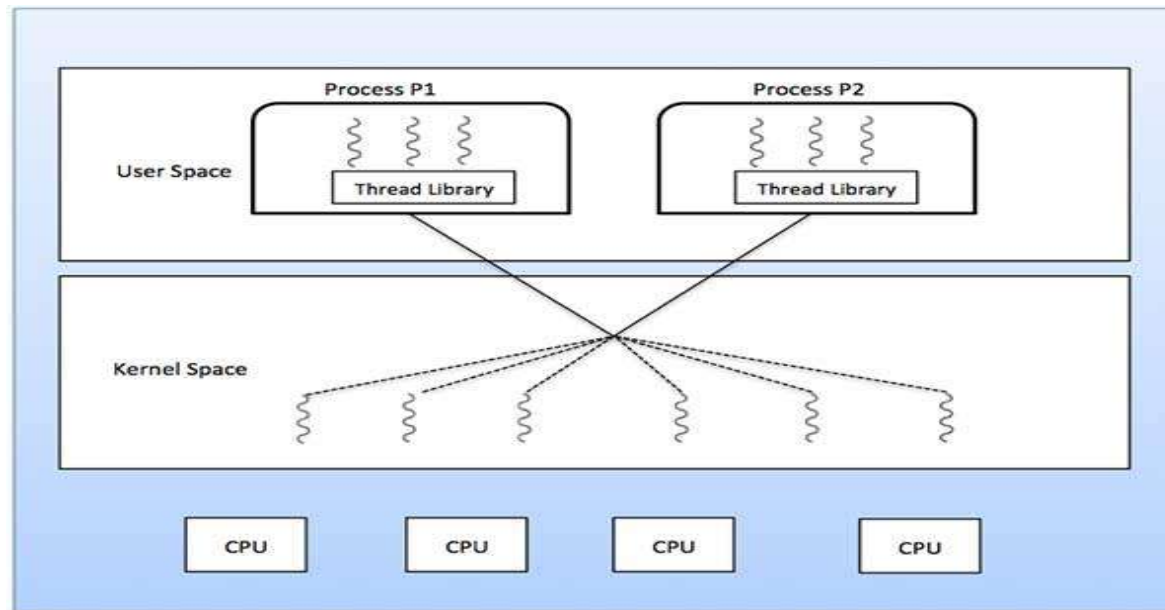
Some operating system provide a combined user level thread and Kernel level thread facility. Solaris is a good example of this combined approach. In a combined system, multiple threads within the same application can run in parallel on multiple processors and a blocking system call need not block the entire process. Multithreading models are three types:

- Many to many relationship.
- Many to one relationship.
- One to one relationship.

Many to Many Model:

The many-to-many model multiplexes any number of user threads onto an equal or smaller number of kernel threads.

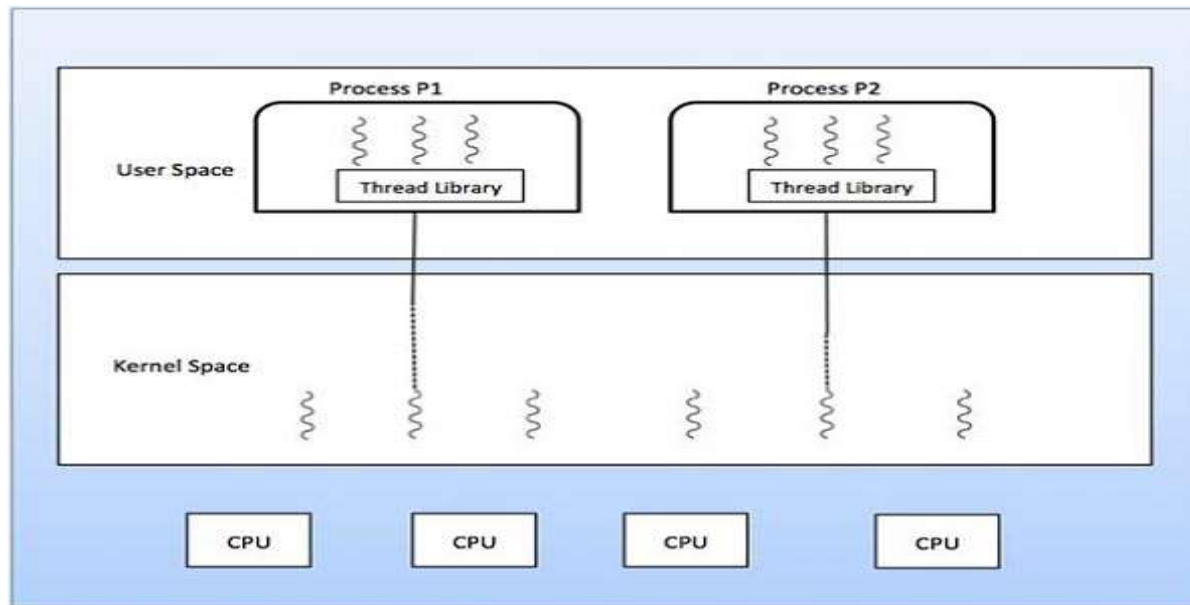
The following diagram shows the many-to-many threading model where 6 user level threads are multiplexing with 6 kernel level threads. In this model, developers can create as many user threads as necessary and the corresponding Kernel threads can run in parallel on a multiprocessor machine. This model provides the best accuracy on concurrency and when a thread performs a blocking system call, the kernel can schedule another thread for execution.



Many to One Model:

Many-to-one model maps many user level threads to one Kernel-level thread. Thread management is done in user space by the thread library. **When thread makes a blocking system call, the entire process will be blocked.** Only one thread can access the Kernel at a time, so multiple threads are unable to run in parallel on multiprocessors.

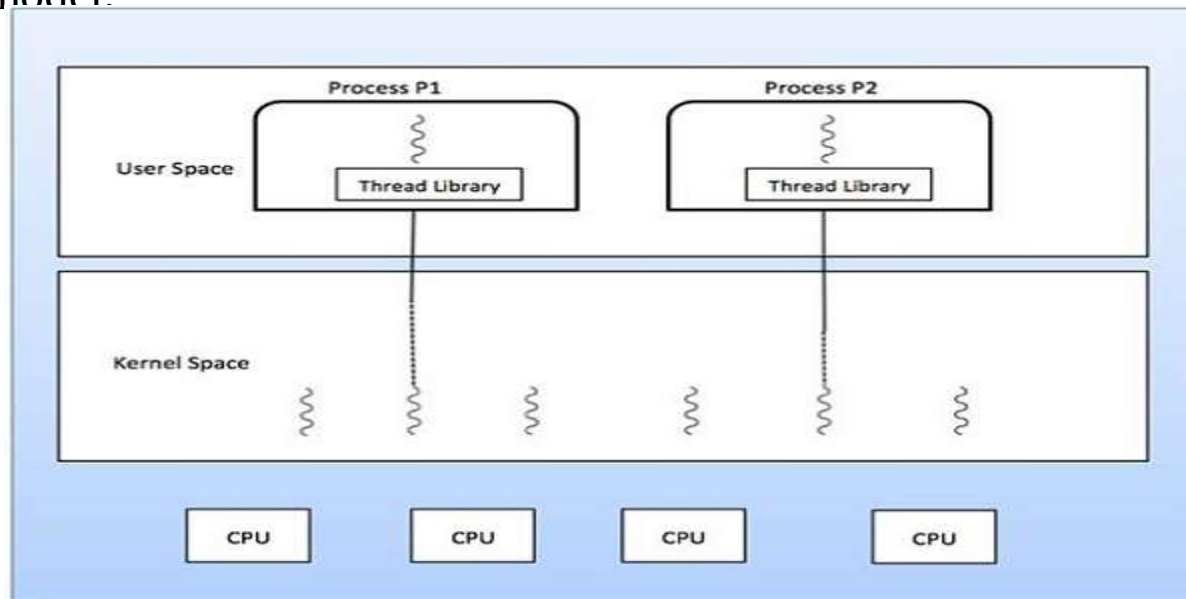
If the user-level thread libraries are implemented in the operating system in such a way that the system does not support them, then the Kernel threads use the many-to-one relationship modes.



One to One Model:

There is one-to-one relationship of user-level thread to the kernel-level thread. This model provides more concurrency than the many-to-one model. It also allows another thread to run when a thread makes a blocking system call. It supports multiple threads to execute in parallel on microprocessors.

Disadvantage of this model is that creating user thread requires the corresponding Kernel thread. OS/2, windows NT and windows 2000 use one to one relationship model.



PROCESS SCHEDULING

The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy.

Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Important reason for using a process scheduling system is that it keeps the CPU busy all the time. This allows you to get the minimum response time for programs.

Process scheduling allows OS to allocate a time interval of CPU execution for each process.

CPU-Bound vs I/O-Bound Processes:

A CPU-bound process requires more CPU time or spends more time in the running state.

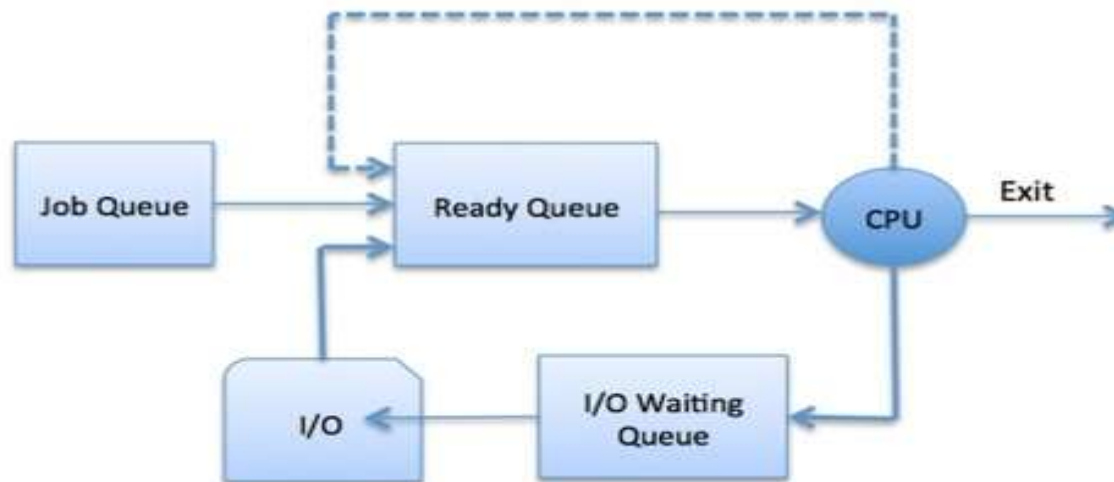
An I/O-bound process requires more I/O time and less CPU time. An I/O-bound process spends more time in the waiting state.

Process Scheduling Queues

The OS maintains all PCBs in Process Scheduling Queues. The OS maintains a separate queue for each of the process states and PCBs of all processes in the same execution state are placed in the same queue. When the state of a process is changed, its PCB is unlinked from its current queue and moved to its new state queue.

The Operating System maintains the following important process scheduling queues –

- **Job queue** – This queue keeps all the processes in the system.
- **Ready queue** – This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- **Device queues** – The processes which are blocked due to unavailability of an I/O device constitute this queue.



The OS can use different policies to manage each queue (FIFO, Round Robin, Priority, etc.). The OS scheduler determines how to move processes between the ready and run queues which can only have one entry per processor core on the system; in the above diagram, it has been merged with the CPU.

Two-State Process Model

Two-state process model refers to running and non-running states which are described below –

S.No.	State & Description
1	Running When a new process is created, it enters into the system as in the running state.
2	Not Running Processes that are not running are kept in queue, waiting for their turn to execute. Each entry in the queue is a pointer to a particular process. Queue is implemented by using linked list. Use of dispatcher is as follows. When a process is interrupted, that process is transferred in the waiting queue. If the process has completed or aborted, the process is discarded. In either case, the dispatcher then selects a process from the queue to execute.

Schedulers



Schedulers are special system software which handle process scheduling in various ways. Their main task is to select the jobs to be submitted into the system and to decide which process to run. Schedulers are of three types –

- Long-Term Scheduler
- Short-Term Scheduler
- Medium-Term Scheduler

Long Term Scheduler

It is also called a **job scheduler**. A long-term scheduler determines which programs are admitted to the system for processing. It selects processes from the queue and loads them into memory for execution. Process loads into the memory for CPU scheduling.

The primary objective of the job scheduler is to provide a balanced mix of jobs, such as I/O bound and processor bound. It also controls the degree of multiprogramming. If the degree of multiprogramming is stable, then the average rate of process creation must be equal to the average departure rate of processes leaving the system.

On some systems, the long-term scheduler may not be available or minimal. Time-sharing operating systems have no long term scheduler. When a process changes the state from new to ready, then there is use of long-term scheduler.

Short Term Scheduler

It is also called as **CPU scheduler**. Its main objective is to increase system performance in accordance with the chosen set of criteria. It is the change of ready state to running state of the process. CPU scheduler selects a process among the processes that are ready to execute and allocates CPU to one of them.

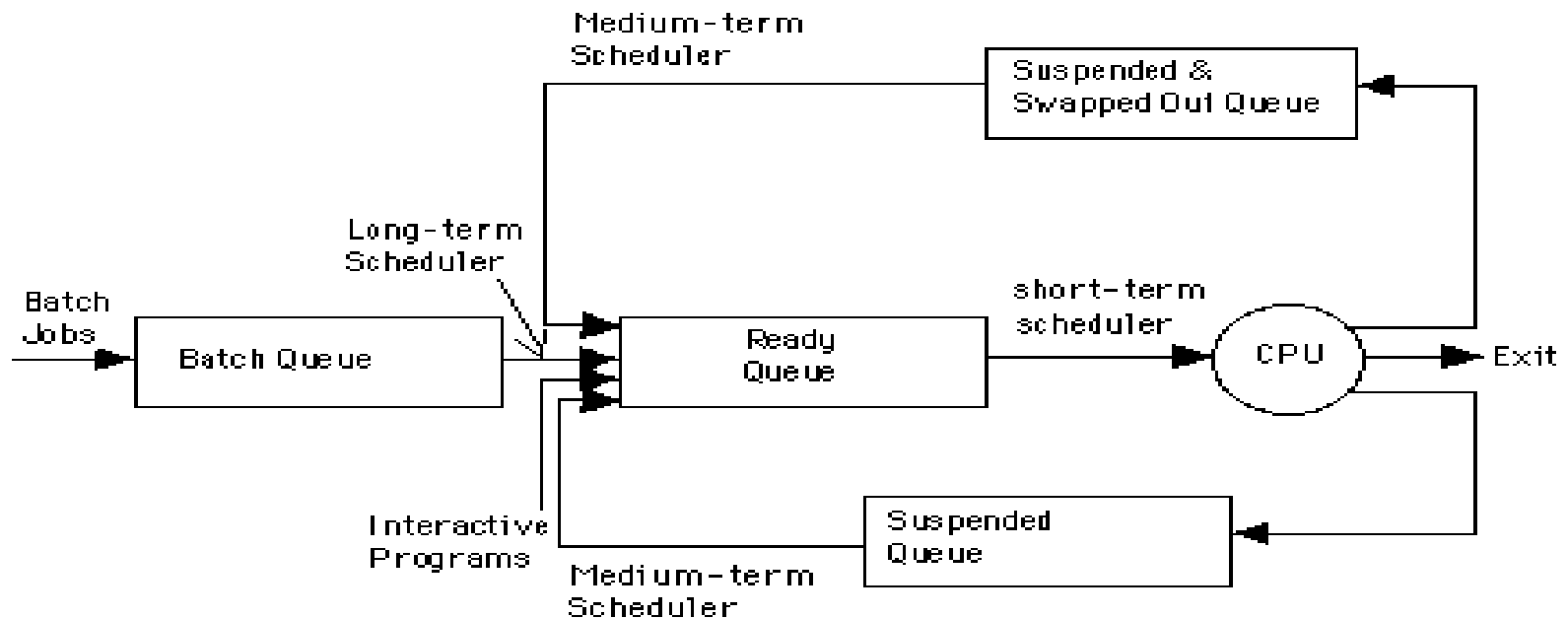
Short-term schedulers, also known as dispatchers, make the decision of which process to execute next. Short-term schedulers are faster than long-term schedulers.

Medium Term Scheduler

Medium-term scheduling is a part of **swapping**. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

A running process may become suspended if it makes an I/O request. A suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called **swapping**, and the process is said to be swapped out or rolled out. Swapping may be necessary to improve the process mix.

The below diagram represents the all the schedulers



Comparison among Scheduler

S. No.	Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
1	It is a job scheduler	It is a CPU scheduler	It is a process swapping scheduler.
2	Speed is lesser than short term scheduler	Speed is fastest among other two	Speed is in between both short and long term scheduler.
3	It controls the degree of multiprogramming	It provides lesser control over degree of multiprogramming	It reduces the degree of multiprogramming.
4	It is almost absent or minimal in time sharing system	It is also minimal in time sharing system	It is a part of Time sharing systems.
5	It selects processes from pool and loads them into memory for execution	It selects those processes which are ready to execute	It can re-introduce the process into memory and execution can be continued.

SCHEDULING CRITERIA

Criteria that help in designing a good scheduling algorithm are

1. CPU Utilisation –

The main objective of any CPU scheduling algorithm is to keep the CPU as busy as possible. Theoretically, CPU utilisation can range from 0 to 100 but in a real-time system, it varies from 40 to 90 percent depending on the load upon the system.

2. Throughput –

A measure of the work done by CPU is the number of processes being executed and completed per unit time. This is called throughput. Throughput is a parameter to measure the performance of CPU. The throughput may vary depending upon the length or duration of processes.

3. Turnaround time –

For a particular process, an important criteria is how long it takes to execute that process. The time elapsed from the time of submission of a process to the time of completion is known as the turnaround time. Turn-around time is the sum of times spent waiting to get into memory, waiting in ready queue, executing in CPU, and waiting for I/O.

4. Waiting time –

A scheduling algorithm does not affect the time required to complete the process once it starts execution. It only affects the waiting time of a process i.e. time spent by a process waiting in the ready queue.

5. Response time –

In an interactive system, turn-around time is not the best criteria. A process may produce some output fairly early and continue computing new results while previous results are being output to the user. Thus another criteria is the time taken from submission of the process of request until the first response is produced. This measure is called response time.

Fairness –

A good scheduler should make sure that each process gets its fair share of the CPU.

SCHEDULING ALGORITHMS

Scheduling Algorithms are Categorized into 2-types. They are

- Non – Preemptive Scheduling
- Pre-emptive scheduling

Non-Preemptive Scheduling:

In this technique, if any resource allocated to a process, the allocated resource should held with the particular process until and unless the process completes its execution. Non-preemptive algorithms are designed so that once a process enters the running state, it cannot be preempted until it completes its allotted time.

Pre-emptive Scheduling:

in this technique, if any resource allocated to a process, the allocated resource can be removed from the process based on the approach(scheduling algorithm) it is following. the preemptive scheduling is based on priority where a scheduler may preempt a low priority running process anytime when a high priority process enters into a ready state.

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms.

The following are the popular process scheduling algorithms, which may either Non-preemptive or Pre-emptive.

- First Come First Serve (FCFS) Scheduling
- Shortest Job First (SJF) Scheduling
- Shortest Remaining Time First (SRTF) Scheduling
- Priority Scheduling
- Round Robin Scheduling
- Multiple - Level Queue Scheduling

- **First Come First Serve (FCFS)**
 - Jobs are executed on first come, first serve basis.
 - It is a non-preemptive scheduling algorithm.
 - Easy to understand and implement.
 - Its implementation is based on FIFO queue.
 - Poor in performance as average wait time is high.

- Example:

Process Id	Arrival time	Burst time
P1	3	4
P2	5	3
P3	0	2
P4	5	1
P5	4	3



Gantt Chart

Here, black box represents the idle time of CPU.

Process Id	Exit time	Turn Around time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 4 = 0$
P2	13	$13 - 5 = 8$	$8 - 3 = 5$
P3	2	$2 - 0 = 2$	$2 - 2 = 0$
P4	14	$14 - 5 = 9$	$9 - 1 = 8$
P5	10	$10 - 4 = 6$	$6 - 3 = 3$

$$\begin{aligned}
 \text{Average Turn Around time} &= (P1+P2+P3+P4+P5)/5 \\
 &= (4 + 8 + 2 + 9 + 6) / 5 \\
 &= 29 / 5 \\
 &= 5.8 \text{ units}
 \end{aligned}$$

$$\begin{aligned}
 \text{Average waiting time} &= (3-3)+(10-5)+(0-0)+(13-5)+(7-4) \\
 &= (0 + 5 + 0 + 8 + 3) / 5 \\
 &= 16 / 5 \\
 &= 3.2 \text{ units}
 \end{aligned}$$

Shortest Job Next (SJN):

- This is also known as **shortest job first**, or SJF
- This is a non-preemptive scheduling algorithm.
- Best approach to minimize waiting time.
- Easy to implement in Batch systems where required CPU time is known in advance.
- Impossible to implement in interactive systems where required CPU time is not known.
- The processor should know in advance how much time process will take.

Example:

Process Id	Arrival time	Burst time
P1	3	1
P2	1	4
P3	4	2
P4	0	6
P5	2	3

Gantt Chart-



Gantt Chart

Process Id	Exit time	Turn Around time	Waiting time
P1	7	$7 - 3 = 4$	$4 - 1 = 3$
P2	16	$16 - 1 = 15$	$15 - 4 = 11$
P3	9	$9 - 4 = 5$	$5 - 2 = 3$
P4	6	$6 - 0 = 6$	$6 - 6 = 0$
P5	12	$12 - 2 = 10$	$10 - 3 = 7$

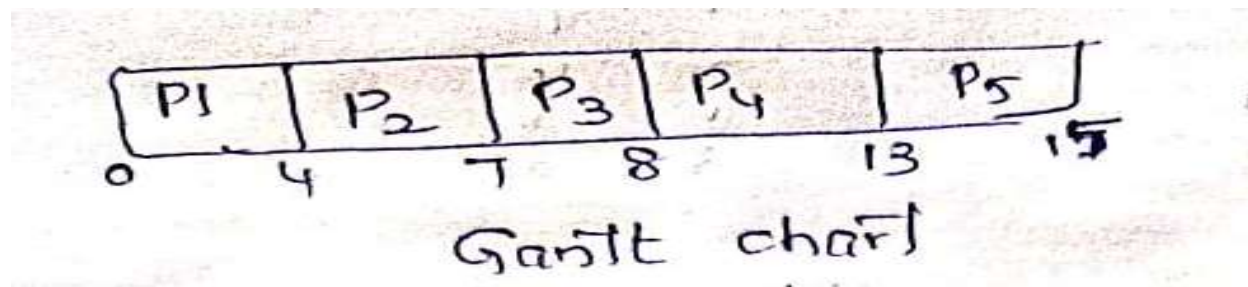
- Average Turn Around time = $(4 + 15 + 5 + 6 + 10) / 5 = 40 / 5 = 8$ unit
- Average waiting time = $(3 + 11 + 3 + 0 + 7) / 5 = 24 / 5 = 4.8$ unit

Priority Scheduling:

- Priority scheduling is a non-preemptive algorithm and one of the most common scheduling algorithms in batch systems.
- Each process is assigned a priority. Process with highest priority (means least number) is to be executed first and so on.
- Processes with same priority are executed on first come first served basis.
- Priority can be decided based on memory requirements, time requirements or any other resource requirement.

Example:

Process Id	Arrival time	Burst time	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5



Process id	Exit time	Turn-Around Time = Exit Time - A.T	Waiting time = T.A.T - B.T.
P ₁	4	4 - 0 = 4	4 - 4 = 0
P ₂	7	7 - 1 = 6	6 - 3 = 3
P ₃	8	8 - 2 = 6	6 - 1 = 5
P ₄	13	13 - 3 = 10	10 - 5 = 5
P ₅	15	15 - 4 = 11	11 - 2 = 9

Now,

$$\text{Average Turn-Around Time} = \frac{4 + 6 + 6 + 10 + 11}{5}$$

$$= \frac{37}{5} = 7.4 \text{ unit}$$

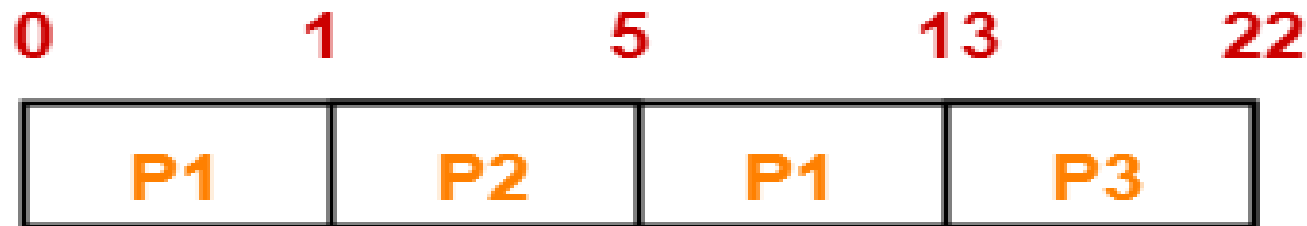
$$\text{Average Waiting Time} = \frac{0 + 3 + 5 + 5 + 9}{5} = \frac{22}{5} = 4.4 \text{ unit}$$

Shortest Remaining Time First / Next

- Shortest remaining time First / Next (SRTF) is the preemptive version of the SJF algorithm.
- The processor is allocated to the job closest to completion but it can be preempted by a newer ready job with shorter time to completion.
- Impossible to implement in interactive systems where required CPU time is not known.
- It is often used in batch environments where short jobs need to give preference.

Example:

Process Id	Arrival time	Burst time
P1	0	9
P2	1	4
P3	2	9



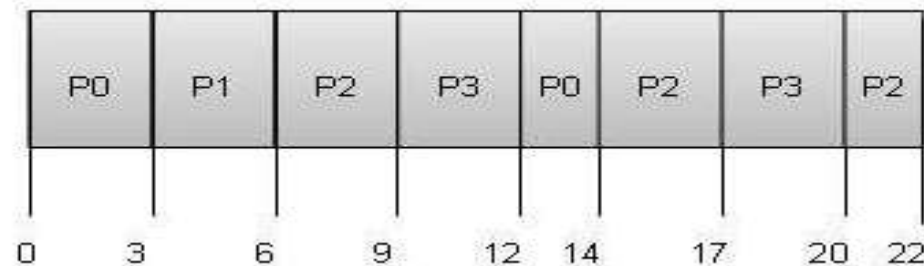
Process Id	Exit time	Turn Around time	Waiting time
P1	13	$13 - 0 = 13$	$13 - 9 = 4$
P2	5	$5 - 1 = 4$	$4 - 4 = 0$
P3	22	$22 - 2 = 20$	$20 - 9 = 11$

Now,

- Average Turn Around time = $(13 + 4 + 20) / 3 = 37 / 3 = 12.33$ unit
- Average waiting time = $(4 + 0 + 11) / 3 = 15 / 3 = 5$ unit

Round Robin Scheduling

- Round Robin is the preemptive process scheduling algorithm.
- Each process is provided a fixed time to execute, it is called a **time quantum or time slice**.
- Once a process is executed for a given time period, it is preempted and other process executes for a given time period.
- Context switching is used to save states of preempted processes.
 - If time quantum=3



Round Robin example

Process Id	Arrival time	Burst time
P1	0	5
P2	1	3
P3	2	1
P4	3	2
P5	4	3

Round Robin with time quantum = 2 unit, calculate the average waiting time and average turn around time.

0 2 4 5 7 9 11 12 13 14

P1	P2	P3	P1	P4	P5	P2	P1	P5
----	----	----	----	----	----	----	----	----

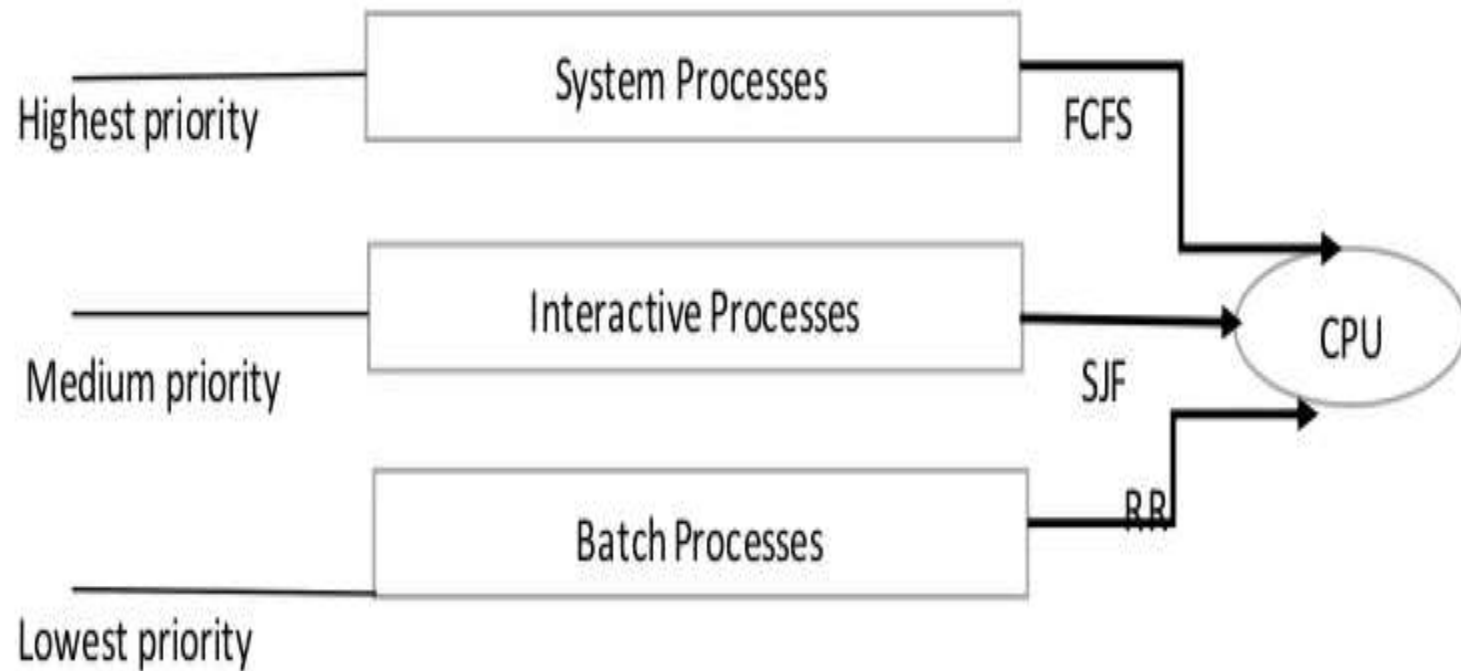
Process Id	Exit time	Turn Around time	Waiting time
P1	13	$13 - 0 = 13$	$13 - 5 = 8$
P2	12	$12 - 1 = 11$	$11 - 3 = 8$
P3	5	$5 - 2 = 3$	$3 - 1 = 2$
P4	9	$9 - 3 = 6$	$6 - 2 = 4$
P5	14	$14 - 4 = 10$	$10 - 3 = 7$

Now,

- Average Turn Around time = $(13 + 11 + 3 + 6 + 10) / 5 = 43 / 5 = 8.6$ unit
- Average waiting time = $(8 + 8 + 2 + 4 + 7) / 5 = 29 / 5 = 5.8$ unit

Multi-Level Queues Scheduling

- Multi-level queues are not an independent scheduling algorithm. They make use of other existing algorithms to group and schedule jobs with common characteristics.
 - Multiple queues are maintained for processes with common characteristics.
 - Each queue can have its own scheduling algorithms.
 - Priorities are assigned to each queue.
- For example, CPU-bound jobs can be scheduled in one queue and all I/O-bound jobs in another queue. The Process Scheduler then alternately selects jobs from each queue and assigns them to the CPU based on the algorithm assigned to the queue.

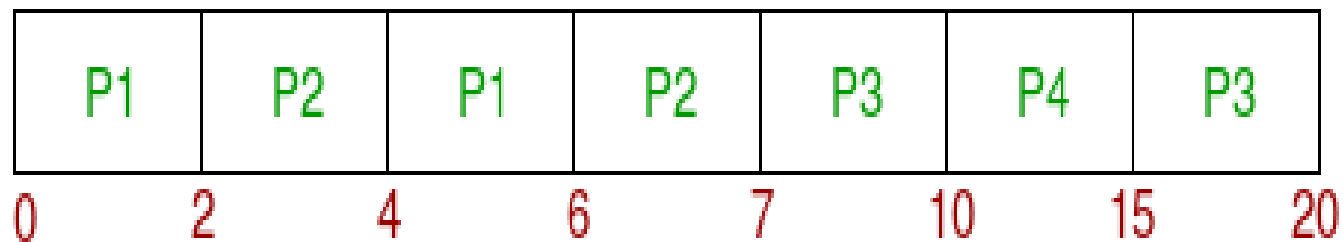


example

Process	Arrival Time	CPU Burst Time	Queue Number
P1	0	4	1
P2	0	3	1
P3	0	8	2
P4	10	5	1

Priority of queue 1 is greater than queue 2. queue 1 uses Round Robin (queue 2 Time Quantum = 2) and uses FCFS.

Below is the **gantt chart** of the problem :



Multiple Processors Scheduling



Multiple processor scheduling or multiprocessor scheduling focuses on designing the system's scheduling function, which consists of more than one processor. Multiple CPUs share the load (load sharing) in multiprocessor scheduling so that various processes run simultaneously. In general, multiprocessor scheduling is complex as compared to single processor scheduling. In the multiprocessor scheduling, there are many processors, and they are identical, and we can run any process at any time.

The multiple CPUs in the system are in close communication, which shares a common bus, memory, and other peripheral devices. So we can say that the system is tightly coupled. These systems are used when we want to process a bulk amount of data, and these systems are mainly used in satellite, weather forecasting, etc.

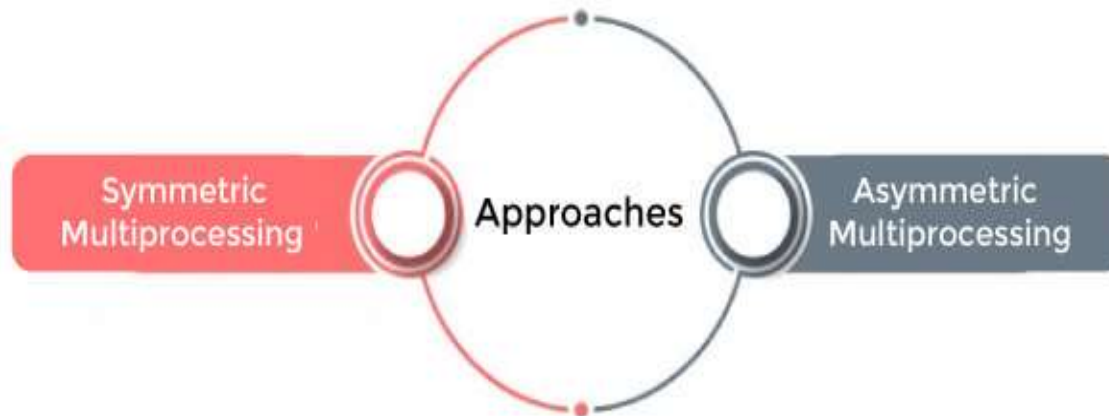
There are cases when the processors are identical, i.e., homogenous, in terms of their functionality in multiple-processor scheduling. We can use any processor available to run any process in the queue.

Multiprocessor systems may be **heterogeneous** (different kinds of CPUs) or **homogenous** (the same CPU). There may be special scheduling constraints, such as devices connected via a private bus to only one CPU.

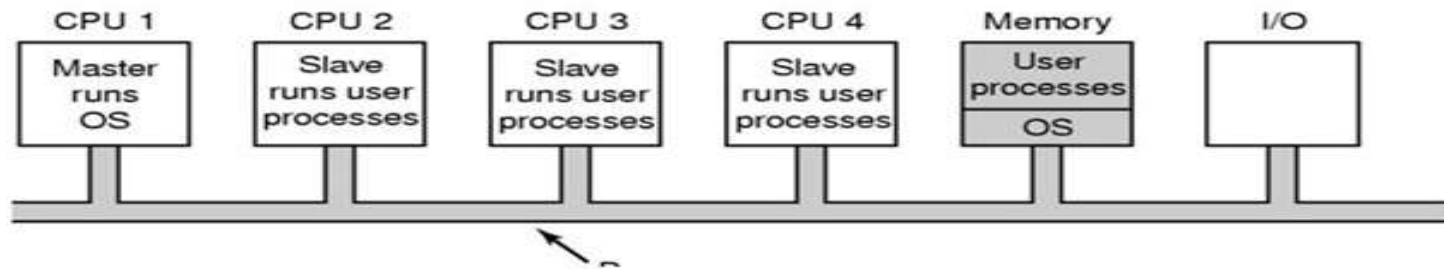
There is no policy or rule which can be declared as the best scheduling solution to a system with a single processor. Similarly, there is no best scheduling solution for a system with multiple processors as well.

Approaches to Multiple Processor Scheduling

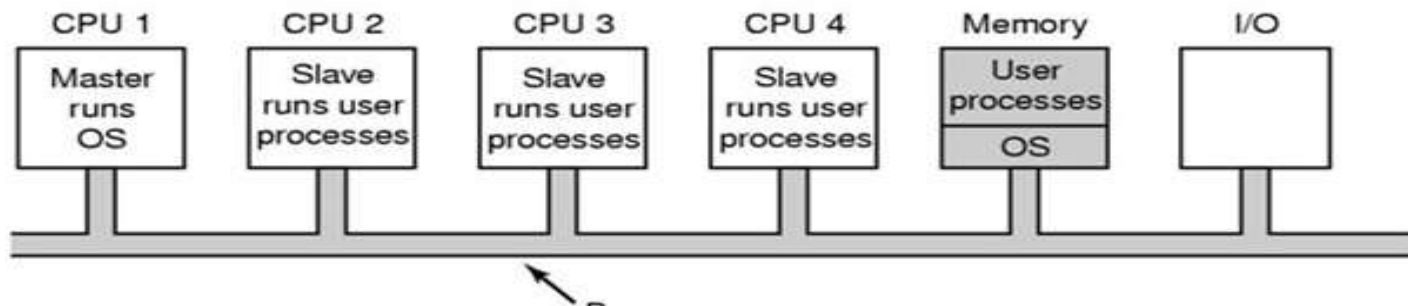
There are two approaches to multiple processor scheduling in the operating system: Symmetric Multiprocessing and Asymmetric Multiprocessing.



1. Asymmetric Multiprocessing: It is used when all the scheduling decisions and I/O processing are handled by a single processor called the **Master Server**. The master server runs the **OS process** and the slave server runs the **user process**. This is simple and reduces the need for data sharing, and this entire scenario is called **Asymmetric Multiprocessing**.



2 Symmetric Multiprocessing: In this, there is one copy of OS in memory, but any central processing unit can run it. It is used where each processor is **self-scheduling**. All processes may be in a common ready queue, or each processor may have its private queue for ready processes. The scheduling proceeds further by having the scheduler for each processor examine the ready queue and select a process to execute.



There are mainly three sources of contention that can be found in a multiprocessor operating system.

- Locking system**

As we know that the resources are shared in the multiprocessor system so there is a need to protect these resources for safe access among the multiple processors. The main purpose of locking scheme is to serialize access of the resources by the multiple processors.

- Shared data**

When the multiple processor access the same data at the same time then there may be a chance of inconsistency of data so to protect this we have to use some protocols or locking scheme.

- Cache coherence**

It is the shared resource data which is stored in the multiple local caches. Suppose there are two clients have a cached copy of memory and one client change the memory block and the other client could be left with invalid cache without notification of the change so this kind of conflict can be resolved by maintaining a coherence view of the data.

Processor Affinity

Processor Affinity means a process has an **affinity** for the processor on which it is currently running. When a process runs on a specific processor, there are certain effects on the cache memory. The data most recently accessed by the process populate the cache for the processor. As a result, successive memory access by the process is often satisfied in the cache memory.

Now, suppose the process migrates to another processor. In that case, the contents of the cache memory must be invalidated for the first processor, and the cache for the second processor must be repopulated. Because of the high cost of invalidating and repopulating caches, most SMP (symmetric multiprocessing) systems try to avoid migrating processes from one processor to another and keep a process running on the same processor. This is known as processor affinity.

Types of Processor Affinity



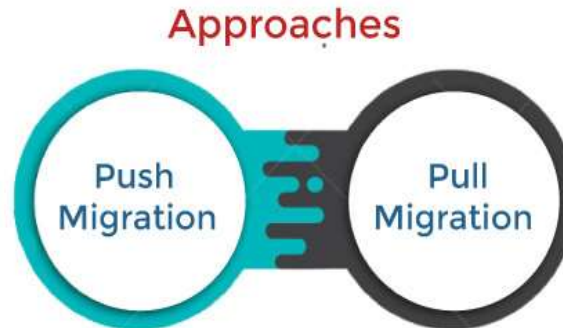
1.Soft Affinity: When an operating system has a policy of keeping a process running on the same processor but not guaranteeing it will do so, this situation is called soft affinity.

2.Hard Affinity: Hard Affinity allows a process to specify a subset of processors on which it may run. Some Linux systems implement soft affinity and provide system calls like ***sched_setaffinity()*** that also support hard affinity.

Load Balancing

Load Balancing is the phenomenon that keeps the workload evenly distributed across all processors in an SMP system. Load balancing is necessary only on systems where each processor has its own private queue of a process that is eligible to execute.

Load balancing is unnecessary because it immediately extracts a runnable process from the common run queue once a processor becomes idle. On SMP (symmetric multiprocessing), it is important to keep the workload balanced among all processors to utilize the benefits of having more than one processor fully. One or more processors will sit idle while other processors have high workloads along with lists of processors awaiting the CPU. There are two general approaches to load balancing:



1. Push Migration:

In push migration, a task routinely checks the load on each processor. If it finds an imbalance, it evenly distributes the load on each processor by moving the processes from overloaded to idle or less busy processors.

2. Pull Migration:

Pull Migration occurs when an idle processor pulls a waiting task from a busy processor for its execution.